

UCT41042 - PRACTICAL FOR INTRODUCTION TO SMART SYSTEMS

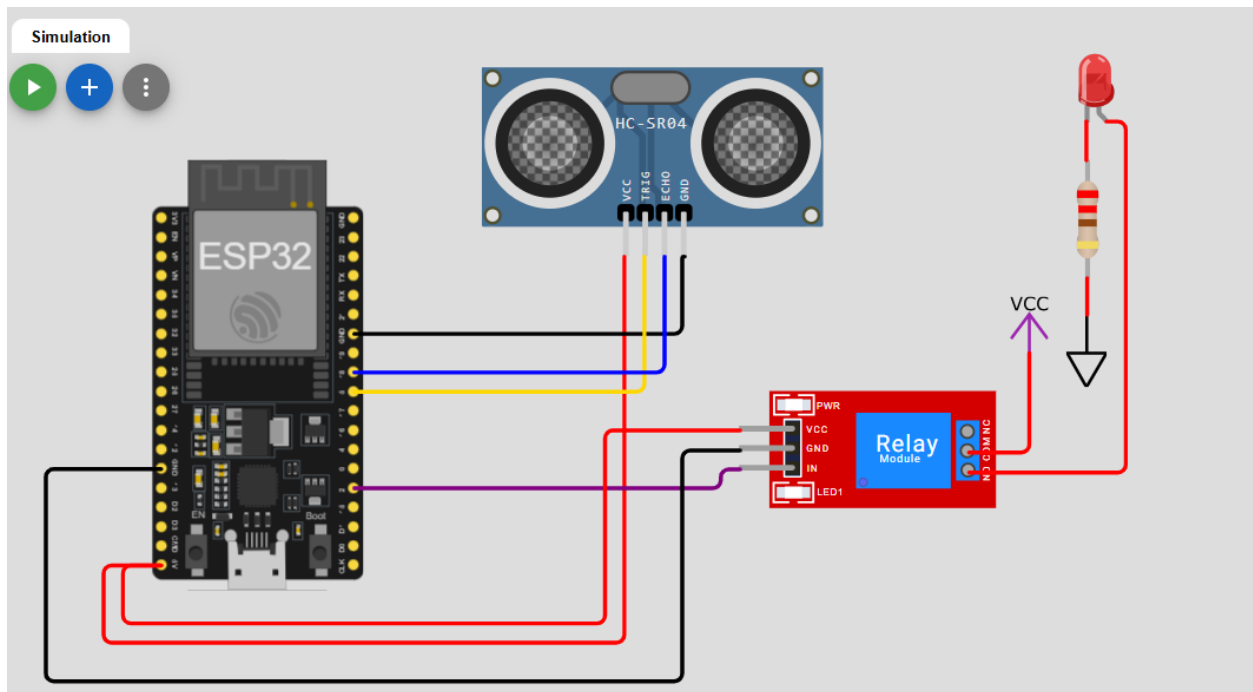
Assignment 02

Reg.No:SEU/IS/20/ICT/084

Name : P.R.P.S.Kalhara

Q01

Question 1 – Circuit



Question 1 – Code

```
#include <Arduino.h>
#include <WiFi.h>
#include <PubSubClient.h>

#define TRIG_PIN 5
#define ECHO_PIN 18
#define PUMP_PIN 2

#define WIFI_SSID "Wokwi-GUEST"
#define WIFI_PASSWORD ""

#define MQTT_BROKER "192.168.116.178"
#define MQTT_PORT 1883
#define MQTT_CLIENT "esp32-water-tank"

WiFiClient espClient;
```

```

PubSubClient mqtt(espClient);

const char *TOPIC_LEVEL = "home/tank/level";
const char *TOPIC_PUMP = "home/tank/pump";
const char *TOPIC_CMD = "home/tank/pump/set";

bool manualMode = false;

void callback(char *topic, byte *payload, unsigned int length){

    String msg = "";
    for (int i = 0; i < length; i++)
        msg += (char)payload[i];
    if (String(topic) == TOPIC_CMD){
        if (msg == "ON"){
            digitalWrite(PUMP_PIN, HIGH);
            mqtt.publish(TOPIC_PUMP, "ON", true);
        }
        if (msg == "OFF"){
            digitalWrite(PUMP_PIN, LOW);
            mqtt.publish(TOPIC_PUMP, "OFF", true);
        }
    }
}

void connectWiFi(){
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    while (WiFi.status() != WL_CONNECTED){
        delay(500);
        Serial.print(".");
    }
    Serial.println("\nWiFi Connected");
}

void connectMQTT(){
    while (!mqtt.connected()){
        Serial.print("Connecting MQTT...");
        if (mqtt.connect(MQTT_CLIENT)){
            Serial.println("Connected");
            mqtt.subscribe(TOPIC_CMD);
        }else{

            delay(2000);
        }
    }
}

```

```

}

float getDistance(){
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);
    long duration = pulseIn(ECHO_PIN, HIGH);
    float distance = duration * 0.034 / 2;
    return distance;
}

void setup(){
    Serial.begin(115200);
    pinMode(TRIG_PIN, OUTPUT);
    pinMode(ECHO_PIN, INPUT);
    pinMode(PUMP_PIN, OUTPUT);
    connectWiFi();
    mqtt.setServer(MQTT_BROKER, MQTT_PORT);
    mqtt.setCallback(callback);
}

void loop(){
    if (!mqtt.connected())
        connectMQTT();
    mqtt.loop();
    static unsigned long lastPub = 0;
    if (millis() - lastPub > 2000){

        float distance = getDistance();

        float level = ((200 - distance) / 200.0) * 100;

        if (level < 0)
            level = 0;
        if (level > 100)
            level = 100;

        Serial.print("Distance: ");
        Serial.print(distance);
        Serial.print(" cm ");

        Serial.print("Level: ");
        Serial.print(level);
    }
}

```

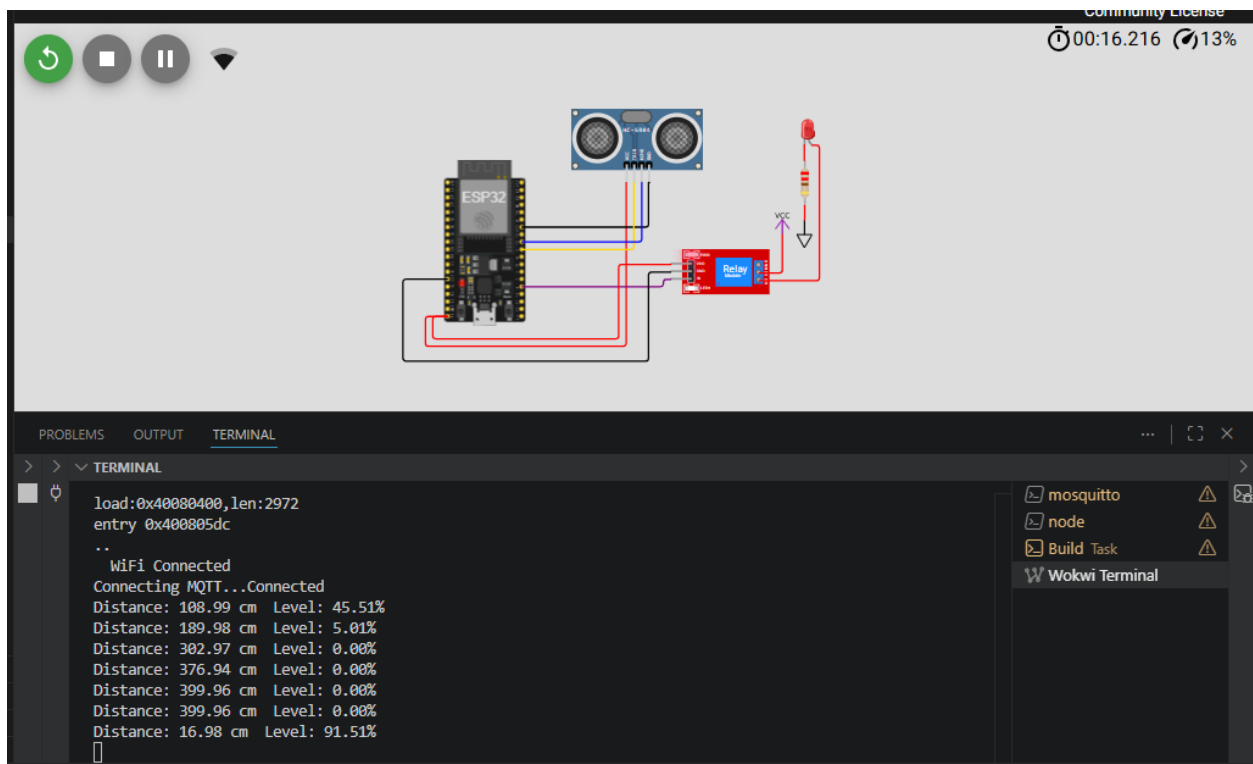
```

Serial.println("%");

mqtt.publish(TOPIC_LEVEL,String((int)level).c_str(),true);
if (level < 30){
    digitalWrite(PUMP_PIN, HIGH);
    mqtt.publish(TOPIC_PUMP, "Pump ON - Low Water Level",true);
}else if (level > 80){
    digitalWrite(PUMP_PIN, LOW);
    mqtt.publish(TOPIC_PUMP,"Tank Full",true);
}else{
    digitalWrite(PUMP_PIN, LOW);
    mqtt.publish(TOPIC_PUMP,"Normal Level",true);
}
lastPub = millis();
}
}

```

Question 1– Serial Monitor



The screenshot shows a Wokwi simulation environment. At the top, there are control buttons (play, stop, pause) and a timer showing 00:16.216 with a 13% battery level. The main workspace displays a circuit diagram with an ESP32 microcontroller, a blue buzzer module, and a red relay module. Wires connect the ESP32 to the buzzer and the relay. A power source is connected to the relay's VCC pin.

Below the workspace is a terminal window with the following output:

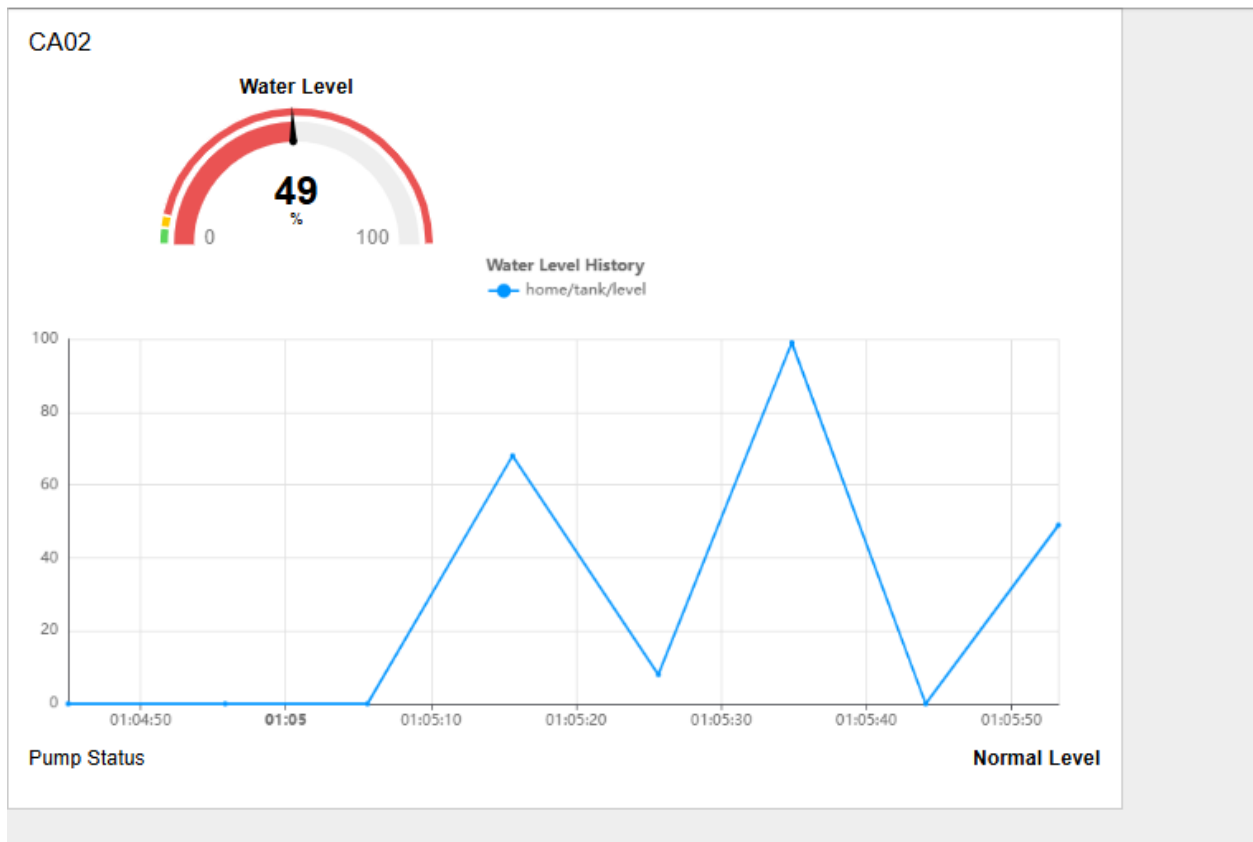
```

load:0x40080400,len:2972
entry 0x400805dc
..
WiFi Connected
Connecting MQTT...Connected
Distance: 108.99 cm Level: 45.51%
Distance: 189.98 cm Level: 5.01%
Distance: 302.97 cm Level: 0.00%
Distance: 376.94 cm Level: 0.00%
Distance: 399.96 cm Level: 0.00%
Distance: 399.96 cm Level: 0.00%
Distance: 16.98 cm Level: 91.51%

```

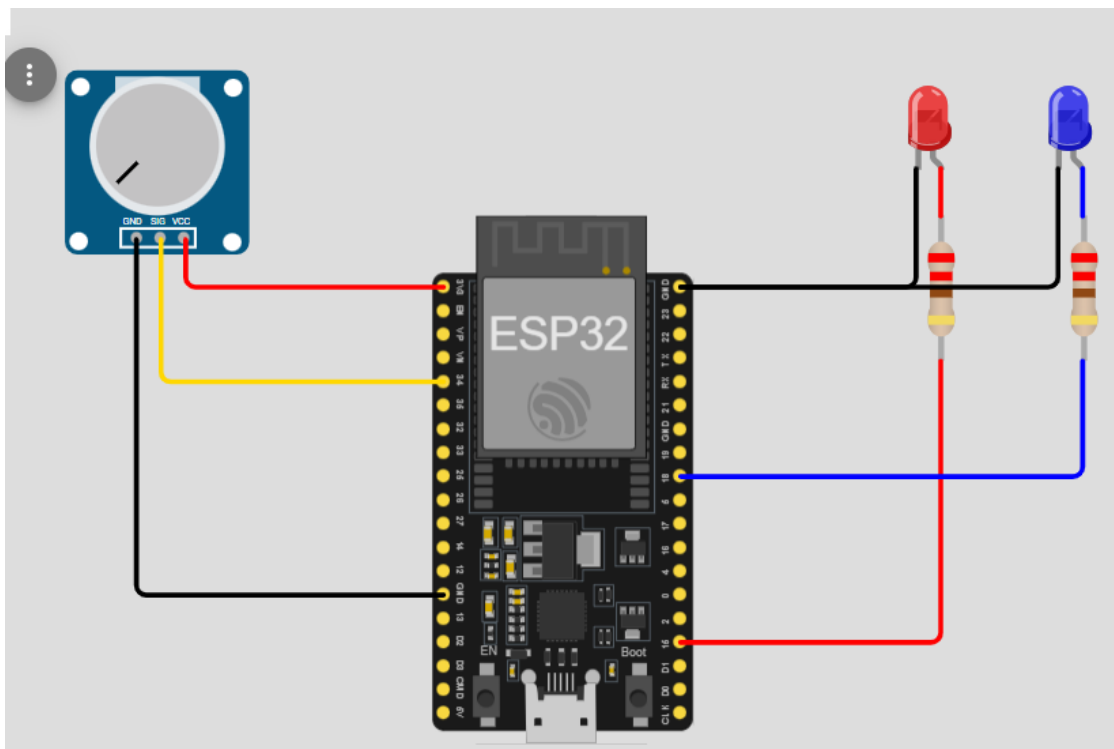
On the right side of the terminal, there is a list of installed packages: mosquitto, node, Build Task, and Wokwi Terminal.

Question 1 – Node-RED Dashboard



Q02

Question 2 – Circuit



Question 2 – Code

```
#include <Arduino.h>
#include <WiFi.h>
#include <PubSubClient.h>

#define POT_PIN 34
#define LOAD1_PIN 15
#define LOAD2_PIN 18

#define WIFI_SSID "Wokwi-GUEST"
#define WIFI_PASSWORD ""

#define MQTT_BROKER "192.168.116.178"
#define MQTT_PORT 1883
#define MQTT_CLIENT "esp32-energy-monitor"

WiFiClient espClient;
PubSubClient mqtt(espClient);

const char *TOPIC_POWER = "home/energy/power";
const char *TOPIC_LOAD1 = "home/energy/load1";
const char *TOPIC_LOAD2 = "home/energy/load2";
const char *TOPIC_ALERT = "home/energy/alert";

const char *TOPIC_LOAD1_CMD = "home/energy/load1/set";
const char *TOPIC_LOAD2_CMD = "home/energy/load2/set";

bool autoShutdown = false;

void callback(char* topic, byte* payload, unsigned int length) {
    String msg="";
    for(int i=0;i<length;i++){
        msg += (char)payload[i];
    }
    if(!autoShutdown){
        if(String(topic)==TOPIC_LOAD1_CMD){
            if(msg=="ON"){
                digitalWrite(LOAD1_PIN,HIGH);
                mqtt.publish(TOPIC_LOAD1,"ON",true);
            }else{
                digitalWrite(LOAD1_PIN,LOW);
                mqtt.publish(TOPIC_LOAD1,"OFF",true);
            }
        }
    }
}
```

```

    }

    if(String(topic)==TOPIC_LOAD2_CMD){
        if(msg=="ON"){
            digitalWrite(LOAD2_PIN,HIGH);
            mqtt.publish(TOPIC_LOAD2,"ON",true);
        }else{
            digitalWrite(LOAD2_PIN,LOW);
            mqtt.publish(TOPIC_LOAD2,"OFF",true);
        }
    }
}

void connectWiFi(){
    WiFi.begin(WIFI_SSID,WIFI_PASSWORD);
    while(WiFi.status()!=WL_CONNECTED){
        delay(500);
        Serial.print(".");
    }
    Serial.println("\nWiFi Connected");
}

void connectMQTT(){
    while(!mqtt.connected()){
        Serial.print("Connecting MQTT...");
        if(mqtt.connect(MQTT_CLIENT)){
            Serial.println("Connected");
            mqtt.subscribe(TOPIC_LOAD1_CMD);
            mqtt.subscribe(TOPIC_LOAD2_CMD);
        }else{
            delay(2000);
        }
    }
}

void setup(){
    Serial.begin(115200);
    pinMode(LOAD1_PIN,OUTPUT);
    pinMode(LOAD2_PIN,OUTPUT);
    connectWiFi();
    mqtt.setServer(MQTT_BROKER,MQTT_PORT);
    mqtt.setCallback(callback);
}

```

```

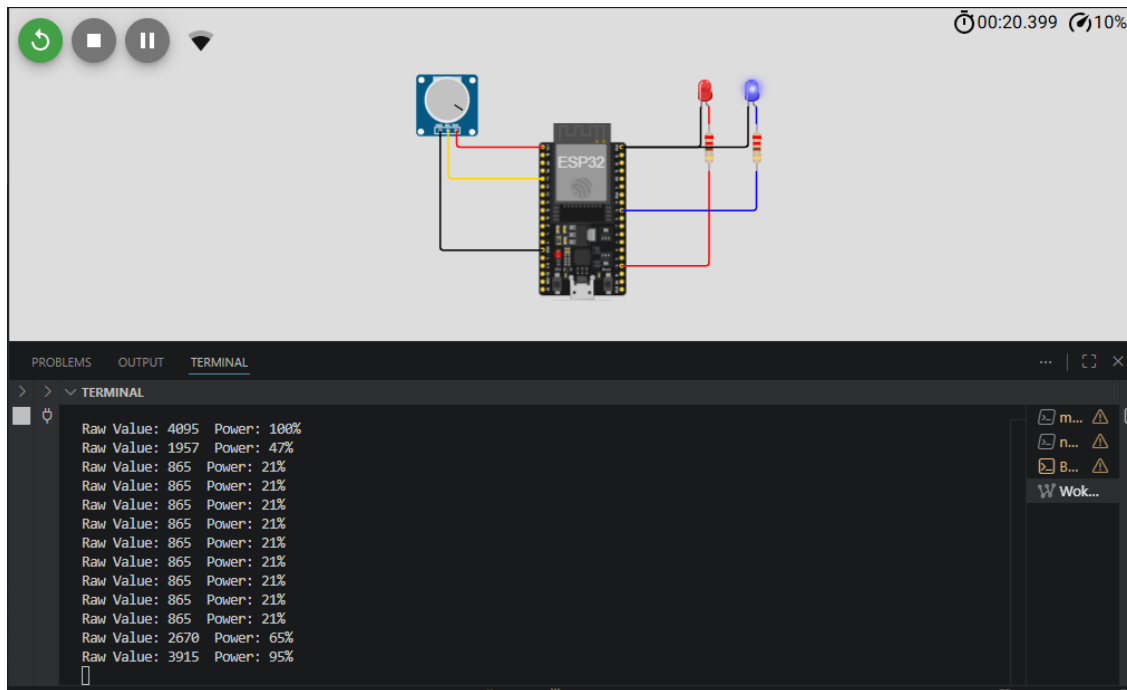
void loop(){
  if(!mqtt.connected()){
    connectMQTT();
  }

  mqtt.loop();
  static unsigned long lastPub=0;
  if(millis()-lastPub > 1000){
    int rawValue = analogRead(POT_PIN);
    int powerPercent = map(rawValue,0,4095,0,100);
    Serial.print("Raw Value: ");
    Serial.print(rawValue);
    Serial.print(" Power: ");
    Serial.print(powerPercent);
    Serial.println("%");
    mqtt.publish(TOPIC_POWER,String(powerPercent).c_str(),true);

    if(powerPercent > 65){
      autoShutdown = true;
      digitalWrite(Load1_PIN,LOW);
      mqtt.publish(TOPIC_LOAD1,"OFF",true);
      mqtt.publish(TOPIC_ALERT,"High Power - Load 1 OFF",true);
    }else{
      autoShutdown = false;
      mqtt.publish(TOPIC_ALERT,"Normal Power Usage",true);
    }
    mqtt.publish(TOPIC_LOAD1,digitalRead(Load1_PIN) ? "ON":"OFF",true);
    mqtt.publish(TOPIC_LOAD2,digitalRead(Load2_PIN) ? "ON":"OFF",true);
    lastPub = millis();
  }
}

```

Question 2– Serial Monitor



Question 2 – Node-RED Dashboard

